
FONDAMENTI DI PROGRAMMAZIONE B

Tempo a disposizione: 2 ore 15 minuti

Nome Cognome Matricola

Esercizio 1 [C++] (15pt). Definire una classe templatica `Set<T>` che realizza il tipo di dato astratto insieme di elementi di tipo `T`. La classe deve definire:

- ▶ un costruttore senza argomenti che crea un insieme vuoto
- ▶ un metodo `void add(T value)` che aggiunge un elemento all'insieme. Se l'elemento è già presente, il metodo deve lanciare un'eccezione
- ▶ un metodo `int size() const` che ritorna la cardinalità dell'insieme
- ▶ un metodo `bool contains(const T& value) const` che verifica se `value` è contenuto nell'insieme;
- ▶ un metodo `Set<T> findInRange(const T& min, const T& max) const` che ritorna un nuovo insieme di valori compresi fra `min` e `max`

Si sovraccarichi l'operatore `<<` in modo tale che stampi gli elementi dell'insieme su uno stream di output `fout` nel formato $\{e_1, e_2, \dots, e_n\}$. Non è consentito utilizzare classi della STL. Se necessario, ridefinire gli opportuni metodi, costruttori e/o operatori. Specificare opportunamente eventuali metodi e parametri costanti. Massimizzare incapsulamento e information hiding.

Esercizio 2 [Java] (15pt). Nel contesto dello sviluppo di un sistema di gestione per gli europei di calcio, si implementino le seguenti classi.

- ▶ Si implementi una classe astratta `Calciatore`. La classe è caratterizzata dal nome, dal cognome e dal numero di maglia del calciatore e deve mettere a disposizione un costruttore che inizializza i campi corrispondenti. Si implementino i metodi `getter` per gli attributi della classe. Inoltre, la classe deve sovrascrivere il metodo `equals` della classe `Object`. Due calciatori sono uguali per il metodo `equals` se hanno lo stesso nome, cognome e numero di maglia. La classe deve implementare l'interfaccia `Comparable<Calciatore>`; il metodo `compareTo` deve confrontare i calciatori in base al numero di maglia. Le classi concrete che estendono `Calciatore` (da non implementare) sono `Portiere`, `Difensore`, `Centrocampista` e `Attaccante`.
- ▶ Si implementi una classe `Squadra`. La classe è caratterizzata dal nome della squadra e dall'insieme di calciatori che fanno parte della squadra. Una squadra non può superare i 22 calciatori, non può avere più di 3 portieri e più di 5 attaccanti. Si implementi un costruttore che prende come parametro un insieme di calciatori, inizializzando il campo corrispondente e assicurando il rispetto delle condizioni imposte. Se una di queste condizioni non è rispettata, deve essere lanciata un'eccezione non controllata. La classe deve sovrascrivere il metodo `equals` della classe `Object`. Due squadre sono uguali per il metodo `equals` se hanno lo stesso nome e gli stessi calciatori.
- ▶ Si implementi la classe `Partita`, caratterizzata da due squadre partecipanti e dalla data della partita (considerare il campo relativo alla data di tipo `String`). Si implementino i metodi `getter` per gli attributi della classe. Per questa classe, non è necessario scrivere il metodo `equals`; si supponga di avere una ridefinizione del metodo `equals` tale che due partite sono uguali per il metodo `equals` se coinvolgono le stesse squadre e hanno la stessa data.
- ▶ Si implementi la classe `Torneo`, caratterizzata dall'insieme delle partite effettuate. La classe deve mettere a disposizione i seguenti metodi:
 - un costruttore senza parametri. Un oggetto della classe, quando costruito, non ha alcuna partita registrata;
 - un metodo `aggiungiPartita` che prende come parametro un oggetto di tipo `Partita` e lo aggiunge all'insieme delle partite effettuate. Se esiste già una partita con le stesse squadre, il metodo deve lanciare un'eccezione di tipo controllato `TorneoException`, da implementare. Inoltre, se esiste un calciatore delle squadre della partita presa come parametro che fa parte anche di una qualsiasi squadra di partite già registrate o dell'altra squadra della partita, il metodo deve lanciare l'eccezione `TorneoException`, descritta precedentemente.
 - un metodo `getPartiteInData` che prende come parametro una data (di tipo `String`) e ritorna l'insieme delle partite che si svolgono in quella data.
- ▶ (+3pt) La classe `Torneo` deve essere iterabile (deve implementare l'interfaccia `Iterable<Partita>`), in modo tale che sia possibile iterare sulle partite registrate.

N.B. Massimizzare incapsulamento e information hiding. Non è richiesta l'implementazione del metodo `hashCode` per le classi richieste. Per ciascuna classe, è possibile supporre di avere a disposizione una implementazione del metodo `hashCode` coerente col metodo `equals` che implementerete.